

文本匹配小案例一：jieba 分词、停用词、技能词标准化

一、这节课要解决什么问题

同学们，先想一个生活场景。

假设你要整理一批快递包裹。每个包裹外面写着一长串信息：

张三，江西省南昌市，电子产品，手机壳，蓝色，已付款。

如果你要快速分拣，就不能只盯着整句话看，而要把它拆成一些有用的信息：

姓名：张三
城市：南昌市
类别：电子产品
商品：手机壳
颜色：蓝色
状态：已付款

简历和岗位匹配也是类似的。

一份简历可能写着：

熟悉 Python、Spark、SQL，做过数据分析项目，了解机器学习。

一个岗位可能写着：

岗位要求掌握 Python 编程，熟悉 Spark 大数据处理，有 SQL 基础。

人类读起来很容易，马上知道它们很相关。但计算机一开始只看到一串字符，它并不知道：

- 哪些是技能词
- 哪些是普通描述
- 哪些词可以忽略
- `python` 和 `Python语言` 是不是同一个技能

所以，在真正计算“简历和岗位有多匹配”之前，我们要先完成三件基础操作：

1. **jieba 分词**：把一大段文本切成一个个词，就像把一张购物小票拆成商品名称、数量、金额。
2. **停用词过滤**：去掉“的、了、和、熟悉、掌握”这类信息量不高的词，就像整理笔记时删掉口头禅和重复废话。
3. **技能词标准化**：把 `py`、`python`、`Python语言` 统一成 `Python`，就像通讯录里把“小王”“王同学”“王明”统一成同一个联系人。

这节课不急着计算相似度。今天的目标是先把文本整理干净，为下一节 TF-IDF 和余弦相似度打基础。

二、学习目标

学完本节后，你应该能够：

1. 知道 NLP 是什么，为什么简历匹配需要 NLP。
2. 说清楚分词、停用词、技能词标准化分别在干什么。
3. 使用 `jieba` 对中文简历和岗位描述进行分词。
4. 编写停用词过滤函数。
5. 编写技能词标准化函数。
6. 把一段简历文本转换成干净的词列表。

7. 初步观察简历和岗位之间有哪些共同关键词。

三、NLP 是什么：让计算机处理人类语言

NLP 是 Natural Language Processing 的缩写，中文叫 **自然语言处理**。

简单说，NLP 就是让计算机处理人类语言的技术。

它可以做很多事：

- 分词：把句子切成词
- 文本分类：判断一段评论是好评还是差评
- 情感分析：判断“这个产品太难用了”是负面情绪
- 关键词提取：从简历里提取 Python 、 Spark 、 SQL
- 文本相似度：判断简历和岗位描述是否接近
- 问答系统：让系统根据文本回答问题
- 大模型对话：让 AI 用自然语言与人交流

我们这门实训不是一上来就做复杂大模型，而是从最基础、最实用的 NLP 技术开始：

原始文本 -> 分词 -> 清洗 -> 标准化 -> 向量化 -> 相似度计算 -> 匹配结果

今天只做前三步：

原始文本 -> 分词 -> 停用词过滤 -> 技能词标准化

四、为什么中文要分词

英文天然有空格：

```
I know Python and Spark
```

切起来很自然：

```
["I", "know", "Python", "and", "Spark"]
```

但中文没有天然空格：

```
我熟悉大数据分析和机器学习
```

这句话可能被切成：

```
["我", "熟悉", "大数据", "分析", "和", "机器学习"]
```

也可能被切成：

```
["我", "熟悉", "大数据分析", "和", "机器学习"]
```

这就像我们整理书架时，要决定：

- 数据分析 是一本书名，还是 数据 和 分析 两个普通词？
- 机器学习 是一个完整概念，还是 机器 和 学习 两个词？

- 自然语言处理 是一个技术方向，不能随便拆得太碎。

所以中文 NLP 的第一步通常是：**分词**。

五、什么是停用词

停用词就是文本中经常出现，但对核心含义帮助不大的词。

例如：

的、了、和、与、或、以及、具有、熟悉、掌握、负责

看两句话：

熟悉 Python 和 Spark
掌握 Python 与 Spark

在简历匹配中，真正重要的是：

Python, Spark

熟悉、掌握、和、与 的作用没有那么大。

可以把停用词理解成整理课堂笔记时删掉的“语气词”和“连接词”。它们让句子更自然，但对提取核心信息帮助不大。

注意：停用词不是“垃圾词”，而是“在当前任务中信息量较低的词”。换一个任务，它们可能又有用了。

六、什么是技能词标准化

简历和岗位描述里，同一个技能可能有很多写法：

原始写法	标准写法
py	Python
python	Python
Python语言	Python
spark	Spark
Apache Spark	Spark
mysql	MySQL
My SQL	MySQL
机器学习	Machine Learning
ML	Machine Learning
自然语言处理	NLP

如果不标准化，系统可能会认为：

py，python，Python语言

是三个不同技能。

生活中也有类似情况：

- 通讯录里 妈妈、老妈、母亲 其实是同一个人。
- 外卖地址里 东华理工大学、东理、学校 可能指同一个地点。
- 商品名里 可乐、Coca-Cola、可口可乐 可能指同一类商品。

技能词标准化的目标就是：

```
["py", "python", "Python语言"] -> ["Python"]
```

这样后续匹配才更准确。

七、环境准备

7.1 安装 jieba

如果还没有安装 jieba，在终端执行：

```
pip install jieba -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com
```

如果在 Ubuntu 虚拟机的实训环境里，请先激活虚拟环境：

```
source ~/ai_env/bin/activate  
pip install jieba -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com
```

7.2 验证安装

新建文件：check_jieba.py

```
# 导入 jieba 库
import jieba

# 准备一句中英文混合文本
text = "我熟悉Python和Spark大数据开发"

# jieba.lcut 会直接返回分词列表
words = jieba.lcut(text)

# 打印分词结果
print(words)
```

运行：

```
python check_jieba.py
```

可能看到：

```
['我', '熟悉', 'Python', '和', 'Spark', '大', '数据', '开发']
```

注意：大数据 有时可能被切成 大 和 数据。这不是你代码写错了，而是分词工具还不一定认识你的专业词。后面我们会用自定义词典改进。

八、案例 1: jieba 基础分词

8.1 本案例目标

先学会最基本的分词操作，观察 jieba 如何处理简历和岗位文本。

8.2 代码清单

文件名: 01_jieba_basic.py

```
# 导入 jieba 分词库
import jieba

# 准备几条简历和岗位相关文本
# 可以把它们理解成几条待整理的信息记录
texts = [
    "熟悉Python、Spark、SQL，做过数据分析项目。",
    "岗位要求掌握Python编程，熟悉Spark大数据处理，有SQL基础。",
    "具有Java、Spring Boot、MySQL后端开发经验。",
    "了解机器学习、深度学习和自然语言处理。"
]

# 遍历每一条文本
for text in texts:
    print("=" * 60)
    print("原始文本: ", text)

    # jieba.lcut 会直接返回一个列表
    # 初学者用 lcut 比 cut 更方便观察结果
    words = jieba.lcut(text)

    print("分词结果: ", words)
```

8.3 运行结果示例

原始文本：熟悉Python、Spark、SQL，做过数据分析项目。

分词结果：['熟悉', 'Python', ',', ' ', 'Spark', ',', ' ', 'SQL', ',', ' ', '做', '过', '数据分析', '项目', '。']

8.4 讲解要点

1. `jieba.lcut(text)` 会把文本切成词列表。
2. 标点符号也会混入结果，后面要清理。
3. 英文技能词一般能保留下来。
4. 分词不是最终结果，只是文本处理第一步。

8.5 课堂练习

把下面的句子加入 `texts`：

熟悉Hadoop、Hive和SparkSQL，参与过实时数仓项目。

观察：

- `Hadoop` 是否被识别？
- `Hive` 是否被识别？
- `SparkSQL` 是否被识别？
- `实时数仓` 有没有被切好？

参考答案：

1. `Hadoop`、`Hive` 这类英文技术词通常会被保留下来。
2. `SparkSQL` 可能会作为一个整体保留，也可能受版本和上下文影响。

- 3. 实时数仓 不一定能被切成一个完整词，可能会被切成 实时 和 数仓 。
- 4. 如果专业词切得不好，可以使用后面讲的 jieba.add_word() 或自定义词典。

九、案例 2：jieba 三种分词模式

9.1 核心概念

jieba 常见分词模式有三种：

模式	方法	特点	适合场景
精确模式	<code>jieba.lcut(text)</code>	尽量切得准确	文本分析
全模式	<code>jieba.lcut(text, cut_all=True)</code>	尽可能切出所有词	了解可能词语
搜索引擎模式	<code>jieba.lcut_for_search(text)</code>	长词会进一步切分	搜索召回

生活类比：

- 精确模式像整理购物清单，只保留最合理的商品项。
- 全模式像把商品名称里可能有用的词全圈出来，信息多但也更乱。
- 搜索引擎模式像在通讯录里既按全名搜，也按简称搜，召回更多结果。

9.2 深度解析：三种模式到底差在哪里

先看一句话：

我熟悉大数据分析、机器学习和自然语言处理

这句话里既有普通词，也有专业词：

- 大数据分析
- 机器学习
- 自然语言处理

分词模式不同，得到的结果会不一样。

1. 精确模式：尽量给出“最合理的一种切法”

精确模式的目标是：

把句子切得比较准确，尽量不要产生太多无关词。

例如它可能会把文本切成：

```
['我', '熟悉', '大数据', '分析', '机器学习', '和', '自然语言', '处理']
```

精确模式不追求“把所有可能词都找出来”，而是追求“这句话正常阅读时应该怎么切”。

这很适合后续文本分析，因为后面要做：

- 停用词过滤；
- 技能词标准化；
- TF-IDF；
- 相似度计算。

如果前面切出太多杂词，后面的相似度就可能被干扰。

2. 全模式：尽可能把所有可能词都切出来

全模式的目标是：

只要看起来可能是词，就尽量切出来。

例如它可能会得到：

['我', '熟悉', '大数', '大数据', '数据', '分析', '机器', '机器学习', '学习', '自然', '自然语言', '语言', '处理']

你会发现，全模式的信息更多，但也更乱。

它的优点是：

- 不容易漏掉可能有用的词；
- 适合观察一句话里可能包含哪些词；
- 适合做初步探索。

它的缺点是：

- 会产生很多重复、细碎、低价值的词；
- 后续匹配时噪声更多；
- 可能让相似度结果变得不稳定。

比如 **机器学习** 已经是一个完整技能词，但全模式还可能额外切出 **机器**、**学习**。如果后续没有处理好，系统可能会误以为这些词也同样重要。

3. 搜索引擎模式：为了“搜得到”，会把长词再切小一点

搜索引擎模式的目标不是做最干净的文本分析，而是：

尽量让用户搜索时能搜到相关内容。

例如一句话里有：

自然语言处理

如果只保留完整词 自然语言处理，那么用户搜索 自然语言 或 语言处理 时，可能搜不到。

搜索引擎模式会倾向于把长词再拆出一些短词，例如：

```
['自然语言处理', '自然', '自然语言', '语言', '处理']
```

这样做的目的就是提高“召回”。

9.3 什么是召回

“召回”是搜索和推荐系统里很重要的概念。

一句话理解：

召回就是：应该被找到的内容，有多少被系统找出来了。

生活例子：

假设你在班级通讯录里找“张伟”，班里其实有 3 个叫张伟的同学。

- 如果系统只找到了 1 个，说明召回不高；
- 如果系统找到了 3 个，说明召回很好；
- 如果系统找到了 3 个张伟，还顺便找出很多不相关的人，召回高，但结果变乱了。

所以召回关注的是：

不要漏掉相关结果。

但只追求召回也有问题，因为系统可能会把很多不相关内容也找出来。

比如岗位要求：

熟悉自然语言处理

学生简历写：

了解语言模型

如果系统能从 自然语言处理 中拆出 语言，就更容易把两段文本联系起来。这就是搜索引擎模式提高召回的意义。

但是对简历匹配来说，如果拆得太碎，也可能出现误判。

例如：

自然语言处理
英语语言表达

这两句话都有 语言，但技术方向完全不同。如果只因为都有 语言 就认为很匹配，就会产生噪声。

所以要同时理解两个概念：

概念	关心什么	例子
召回	相关内容有没有被找出来	3 个相关岗位找到了几个
噪声	找出来的内容里有没有很多无关项	是否混入了不相关岗位

搜索引擎模式通常召回更高，但噪声也可能更多。

9.4 三种模式如何选择

在本课程中，可以这样理解：

任务	推荐模式	原因
观察一句话里可能有哪些词	全模式	能看到更多可能词
做正式文本预处理	精确模式	结果更稳定，噪声更少
做搜索框、关键词检索	搜索引擎模式	更容易搜到相关内容

如果是本节课后面的简历岗位匹配项目，建议默认使用：

```
# 使用精确模式进行分词，这是本项目默认推荐写法
jieba.lcut(text)
```

也就是精确模式。

如果后续要做“输入关键词搜索简历”或“搜索岗位”，可以再考虑搜索引擎模式。

9.5 代码清单

文件名：02_jieba_modes.py


```
import jieba

# 准备一条包含复合词的文本
# 这些词中既有普通词，也有专业词，适合观察不同模式的差异
text = "我熟悉大数据分析、机器学习和自然语言处理"

# 打印原始文本，方便和后面的分词结果对比
print("原始文本：", text)
print("=" * 60)

# 精确模式：最常用，适合后续文本分析
# 它会尽量给出一组比较合理、比较稳定的切分结果
accurate_words = jieba.lcut(text)
print("精确模式：", accurate_words)

# 全模式：会切出更多可能词语，但可能有重复和冗余
# cut_all=True 表示“尽可能多地切词”
# 这种模式适合观察候选词，但不适合直接用于后续相似度计算
full_words = jieba.lcut(text, cut_all=True)
print("全模式：", full_words)

# 搜索引擎模式：适合需要提高召回率的场景
# 它会把较长的词再切成一些较短的词
# 这样用户搜索短词时，也更容易找到包含长词的文本
search_words = jieba.lcut_for_search(text)
print("搜索引擎模式：", search_words)
```

9.6 本项目推荐

本项目建议默认使用 **精确模式**：

```
# 对文本使用精确模式分词，并把结果保存到 words 中
words = jieba.lcut(text)
```

原因：后续要计算 TF-IDF 和余弦相似度，词语太多、太碎、太乱，会让匹配结果变得不稳定。

思考题：为什么不直接使用全模式？

参考答案：

全模式会切出尽可能多的词，虽然召回更多，但噪声也更多。对于简历岗位匹配来说，我们更希望词语准确、稳定。如果切出大量无义词，后续 TF-IDF 和相似度计算可能被干扰。

十、案例 3：清理标点符号和空白字符

10.1 为什么要清理

分词后可能会出现：

```
, . 、 / ; : 空格 换行
```

这些符号就像表格里的空行、重复分隔线、无意义备注，对提取核心技能帮助不大。

10.2 代码清单

文件名：03_clean_tokens.py

```
import jieba

# 原始文本中包含中文标点、英文标点和空格
# 本案例的目标是：分词之后，把这些不适合作为关键词的符号过滤掉
text = "熟悉 Python、Spark、SQL，做过数据分析项目；了解机器学习。"

# 定义需要删除的标点符号集合
# 使用 set 集合的好处是判断速度快
punctuations = {
    ", ", "。", "、", ";", ":", "!", "?",
    ",", ". ", ";", ":", "!", "?",
    "(", ")", " (", ") ", "[", "]", "【", "】",
    " ", "\n", "\t"
}

# 先分词
# raw_words 是“未经清理”的原始分词结果
raw_words = jieba.lcut(text)

# 保存清理后的词
# clean_words 中只保留真正有分析价值的词
clean_words = []

for word in raw_words:
    # 去掉词语两侧的空格
    word = word.strip()

    # 如果是空字符串，跳过
    if not word:
        continue

    # 如果是标点符号，跳过
```

```
if word in punctuations:
    continue

# 保留下来的词加入结果列表
clean_words.append(word)

# 对比打印：让学生看到清理前后有什么变化
print("原始分词：", raw_words)
print("清理结果：", clean_words)
```

10.3 讲解要点

1. 分词后通常要清洗。
2. `strip()` 可以去掉两侧空白。
3. `continue` 表示跳过当前词，继续处理下一个词。
4. 标点对技能匹配帮助不大，可以先过滤。

思考题：所有符号都应该删除吗？

参考答案：

不一定。大多数普通标点可以删除，但有些符号可能有意义。例如 `C++` 中的 `++` 不能随便删，`Node.js` 中的点也可能是技能名的一部分。所以真实项目中要根据技能词特点谨慎处理。

十一、案例 4：停用词过滤

11.1 停用词不是“垃圾词”

停用词不是绝对没用，而是在当前任务里信息量较低。

比如：

熟悉 Python 和 Spark
掌握 Python 与 Spark

在“技能匹配”任务中，重点是：

Python, Spark

所以 熟悉、掌握、和、与 可以暂时过滤。

但如果你以后做“能力强弱判断”，熟悉 和 精通 可能又有意义了。

11.2 代码清单

文件名：04_stopwords_demo.py

```
import jieba

# 准备一段简历/岗位中常见的描述
# 里面既有技能词 Python、Spark、数据分析，也有“熟悉、具有、相关、经验”等弱信息词
text = "熟悉 Python 和 Spark，具有数据分析相关项目经验。"

# 停用词集合
# 小案例中先直接写在代码里，后面会改成 stopwords.json 配置文件
stopwords = {
    "的", "了", "和", "与", "或", "以及",
    "熟悉", "掌握", "具有", "相关", "经验",
    "负责", "参与", "能够", "进行"
}

# 标点符号集合
# 这些符号不适合作为后续匹配关键词
punctuations = {"", " ", "。", "、", "，", "，", "。", " ", "\n", "\t"}

# 分词
# raw_words 是 jieba 刚切出来的原始结果，还没有过滤停用词和标点
raw_words = jieba.lcut(text)

# filtered_words 用来保存过滤后的有效词
filtered_words = []

for word in raw_words:
    # 去掉词语两侧空格，例如 " Python " 变成 "Python"
    word = word.strip()

    # 过滤空词
    if not word:
        continue
```

```
# 过滤标点
if word in punctuations:
    continue

# 过滤停用词
if word in stopwords:
    continue

# 能走到这里，说明这个词不是空词、不是标点、不是停用词
filtered_words.append(word)

# 打印三组结果，帮助学生观察“过滤”到底改变了什么
print("原始文本: ", text)
print("分词结果: ", raw_words)
print("过滤后: ", filtered_words)
```

11.3 运行结果示例

过滤后: ['Python', 'Spark', '数据分析', '项目']

11.4 课堂讨论

问题 1: 经验 要不要过滤?

参考答案: 如果当前任务只关注技能匹配, 经验 可以过滤; 如果后续要判断工作年限或经历丰富程度, 就不应该简单过滤。

问题 2: 项目 要不要过滤?

参考答案: 要看任务。如果几乎每份简历都有“项目”, 它的信息量较低, 可以过滤; 但如果要区分“项目经验”和“理论学习”, 也可以保留。

问题 3: `数据分析` 是不是比 `分析` 更有价值?

参考答案: 是的。 `数据分析` 是更完整的专业能力词, 比单独的 `分析` 更具体, 更适合用于岗位匹配。

问题 4: 停用词过滤过多会不会丢失信息?

参考答案: 会。停用词表不是越大越好, 过滤过多可能误删重要信息。建议先用小样例测试, 再逐步调整。

十二、案例 5: 从文件读取停用词

12.1 为什么停用词要放文件里

如果停用词写死在代码里, 后期维护很不方便。

更好的做法是新建 `stopwords.json` :

```
[ "的", "了", "和", "与", "或", "以及", "熟悉", "掌握", "具有", "相关", "经验" ]
```

这里的 `[]` 表示一个列表。也就是说, `stopwords.json` 本质上还是保存了一组停用词, 只是用 JSON 格式来管理。

这样做有三个好处:

1. 停用词和 Python 代码分开, 后期维护更方便;
2. 和后面的 `skill_alias.json` 风格统一;
3. 以后如果要扩展成分类停用词, 也可以继续使用 JSON。

本案例先使用最简单的一行数组形式, 不做复杂分类, 避免增加初学负担。

12.2 代码清单

文件名: 05_load_stopwords.py

```
import json
import jieba

def load_stopwords(file_path):
    """
    从 JSON 文件中读取停用词。

    stopwords.json 中保存的是一个列表，例如：
    ["的", "了", "和", "熟悉", "掌握"]

    返回 set 集合，方便快速判断某个词是否属于停用词。
    """
    # 打开 JSON 文件
    # file_path 可以是 "stopwords.json", 也可以是其他路径
    with open(file_path, "r", encoding="utf-8") as f:
        # json.load 会把 JSON 数组读取成 Python 列表
        words = json.load(f)

    # json.load(f) 读出来的是 list
    # 转成 set 后，判断某个词是否属于停用词会更快
    return set(words)

text = "熟悉 Python 和 Spark，具有数据分析相关项目经验。"

# 从文件中加载停用词
# 注意：运行代码前，要保证当前目录下已经有 stopwords.json
stopwords = load_stopwords("stopwords.json")

# 标点符号集合，后面用于过滤分词结果中的符号
punctuations = {",", " ", "。", "、", " ", " ", "。", " ", " ", "\n", "\t"}
```

```
# 对文本进行分词
words = jieba.lcut(text)

# 保存最终过滤结果
result = []

for word in words:
    # 每个词先去除首尾空白
    word = word.strip()

    # 空字符串没有分析价值，跳过
    if not word:
        continue

    # 标点符号不是关键词，跳过
    if word in punctuations:
        continue

    # 停用词信息量较低，跳过
    if word in stopwords:
        continue

    # 剩下的词加入结果
    result.append(word)

# 打印停用词表和过滤结果，方便确认配置文件是否生效
print("停用词表：", stopwords)
print("过滤结果：", result)
```

12.3 教学提示

这是一种很重要的工程习惯：**配置和代码分离**。

以后技能词映射、权重参数、路径配置，也可以用类似方式管理。

例如后面的技能词标准化也可以使用 JSON：

```
{  
  "py": "Python",  
  "python": "Python",  
  "Apache Spark": "Spark"  
}
```

思考题：为什么读取停用词时要使用 `set`，而不是 `list`？

参考答案：

因为我们要频繁判断某个词是否在停用词表中。`set` 的查找速度通常比 `list` 更快，适合做“是否存在”的判断。

十三、案例 6：技能词标准化

13.1 标准化前后的区别

原始技能词可能是：

```
py, python, Python语言, spark, Apache Spark, mysql, My SQL
```

标准化后希望变成：

```
Python, Spark, MySQL
```

生活中也常见这种问题：

- 通讯录里的 小李 、 李同学 、 李明 可能是同一个人。
- 商品名里的 可乐 、 可口可乐 、 Coca-Cola 可能属于同一类商品。
- 地址里的 东华理工 、 东华理工大学 、 东理 可能指同一个地方。

13.2 代码清单

文件名： 06_skill_normalize_demo.py

```
# 技能词标准化映射表
# key 是可能出现的原始写法
# value 是我们希望统一成的标准写法
SKILL_MAPPING = {
    "py": "Python",
    "python": "Python",
    "Python语言": "Python",
    "spark": "Spark",
    "Apache Spark": "Spark",
    "mysql": "MySQL",
    "my sql": "MySQL",
    "My SQL": "MySQL",
    "机器学习": "Machine Learning",
    "ML": "Machine Learning",
    "深度学习": "Deep Learning",
    "DL": "Deep Learning",
    "自然语言处理": "NLP",
    "NLP": "NLP"
}
```

```
def normalize_skill(word):
    """
    将单个技能词转换为标准写法。
    如果映射表中存在，就返回标准写法。
    如果不存在，就返回原词。
    """

    # 去除两端空白
    word = word.strip()

    # 小写形式用于处理英文大小写差异
    lower_word = word.lower()
```

```
# 先尝试用原词匹配
if word in SKILL_MAPPING:
    return SKILL_MAPPING[word]

# 再尝试用小写形式匹配
if lower_word in SKILL_MAPPING:
    return SKILL_MAPPING[lower_word]

# 如果没有找到映射, 返回原词
return word

raw_skills = ["py", "python", "Python语言", "spark", "Apache Spark", "mysql", "机器学习"]

# 保存标准化后的技能词
normalized_skills = []

for skill in raw_skills:
    # 对每一个原始技能词调用标准化函数
    # 例如 py -> Python, Apache Spark -> Spark
    normalized_skills.append(normalize_skill(skill))

# 对比原始写法和标准写法
print("原始技能: ", raw_skills)
print("标准化后: ", normalized_skills)
```

13.3 运行结果示例

原始技能: ['py', 'python', 'Python语言', 'spark', 'Apache Spark', 'mysql', '机器学习']
标准化后: ['Python', 'Python', 'Python', 'Spark', 'Spark', 'MySQL', 'Machine Learning']

13.4 进一步去重

标准化后可能出现重复:

```
['Python', 'Python', 'Python', 'Spark']
```

如果直接用集合:

```
# set 可以快速去重  
# 但 set 是无序集合, 转换回 list 后, 词语顺序可能发生变化  
unique_skills = list(set(normalized_skills))
```

可以去重, 但顺序可能被打乱。

如果想保留原顺序, 可以这样写:


```
# 创建一个空列表，用来保存去重后的技能词
unique_skills = []

for skill in normalized_skills:
    # 如果这个技能还没有出现过，就加入结果
    # 如果已经出现过，就跳过，从而实现去重
    if skill not in unique_skills:
        unique_skills.append(skill)

# 这种写法可以保留原来的先后顺序
print(unique_skills)
```

思考题：为什么技能词标准化后还要去重？

参考答案：

因为多个不同写法可能会被标准化成同一个技能。例如 `py`、`python`、`Python语言` 都会变成 `Python`。如果不去重，后续统计或匹配时可能把同一个技能重复计算，影响结果。

十四、案例 7：封装成函数

14.1 为什么要封装

如果所有逻辑都写在一个文件里，后面会越来越乱。

合理的做法是分工：

- `load_stopwords()`：负责加载停用词
- `normalize_skill()`：负责技能标准化

- `preprocess_text()`：负责完整文本预处理

这就像做饭时先把洗菜、切菜、炒菜分成不同步骤。每个步骤清晰，整个流程就不容易乱。

14.2 代码清单

文件名： `07_preprocess_functions.py`

```
import json
import jieba
```

```
def load_stopwords(file_path):
    """
    从 stopwords.json 中读取停用词。
    """
    # 打开 JSON 配置文件
    with open(file_path, "r", encoding="utf-8") as f:
        # 把 JSON 数组读取成 Python 列表
        words = json.load(f)

    # 转成 set, 方便后面快速判断 word in stopwords
    return set(words)
```

```
SKILL_MAPPING = {
    "py": "Python",
    "python": "Python",
    "Python语言": "Python",
    "spark": "Spark",
    "apache spark": "Spark",
    "sql": "SQL",
    "mysql": "MySQL",
    "机器学习": "Machine Learning",
    "深度学习": "Deep Learning",
    "自然语言处理": "NLP"
}
```

```
def normalize_skill(word):
```

```
"""
```

```
    标准化技能词。
```

```
"""
```

```
# 去掉词语两侧空格
```

```
word = word.strip()
```

```
# 准备一个小写版本，用来处理英文大小写差异
```

```
# 例如 Python、python 都可以统一识别
```

```
lower_word = word.lower()
```

```
# 第一种情况：原始写法能直接在映射表中找到
```

```
if word in SKILL_MAPPING:
```

```
    return SKILL_MAPPING[word]
```

```
# 第二种情况：原始写法找不到，但小写后能找到
```

```
if lower_word in SKILL_MAPPING:
```

```
    return SKILL_MAPPING[lower_word]
```

```
# 第三种情况：映射表中没有这个词，说明暂时不需要标准化
```

```
# 直接返回原词
```

```
return word
```

```
def preprocess_text(text, stopwords):
```

```
    """
```

```
    对文本进行预处理：
```

```
    1. jieba 分词
```

```
    2. 去除空词
```

```
    3. 去除标点符号
```

```
    4. 去除停用词
```

```
    5. 技能词标准化
```

```
    6. 去重并保留顺序
```

```

"""
punctuations = {
    ", ", ". ", "\ ", "; ", ": ", "! ", "? ",
    ", ", ". ", "; ", ": ", "! ", "? ",
    "( ", ") ", " ( ", " ) ", "[ ", " ] ", " 【 ", " 】 ",
    " ", "\n", "\t"
}

# 第一步: 使用 jieba 对原始文本进行分词
raw_words = jieba.lcut(text)

# result 用来保存最终的干净词列表
result = []

for word in raw_words:
    # 第二步: 去除词语两端空白
    word = word.strip()

    # 第三步: 过滤空词
    if not word:
        continue

    # 第四步: 过滤标点符号
    if word in punctuations:
        continue

    # 第五步: 过滤停用词
    if word in stopwords:
        continue

    # 第六步: 技能词标准化
    # 例如 py -> Python, apache spark -> Spark

```

```

        word = normalize_skill(word)

        # 第七步：去重并保留顺序
        # 如果这个词已经出现过，就不再重复加入
        if word not in result:
            result.append(word)

    # 返回最终可用于后续匹配计算的词列表
    return result

if __name__ == "__main__":
    # 加载停用词配置
    stopwords = load_stopwords("stopwords.json")

    # 准备一段简历文本和一段岗位文本
    resume_text = "熟悉py、Spark、SQL，具有数据分析相关项目经验。"
    job_text = "岗位要求掌握Python编程，熟悉Apache Spark大数据处理，有SQL基础。"

    # 分别对简历和岗位做同样的预处理
    resume_words = preprocess_text(resume_text, stopwords)
    job_words = preprocess_text(job_text, stopwords)

    # 打印处理结果，方便观察函数是否按预期工作
    print("简历原文：", resume_text)
    print("简历处理结果：", resume_words)

    print("岗位原文：", job_text)
    print("岗位处理结果：", job_words)

```

14.3 教学提示

这个案例已经很接近真实项目中的 `preprocess.py`。

后续完整项目中，可以把这些函数作为文本预处理模块使用。

思考题：为什么要把预处理逻辑封装成函数？

参考答案：

封装函数可以让代码更清晰、更容易复用、更容易测试。后续处理 1 条文本、100 条文本、10000 条文本，都可以调用同一个函数，不需要重复写代码。

十五、案例 8：自定义 jieba 词典

15.1 为什么需要自定义词典

jieba 不一定认识所有专业词。

例如：

```
大数据开发  
数据仓库  
机器学习  
自然语言处理  
SparkSQL
```

如果分词效果不好，后续匹配就可能不准。

15.2 代码清单

文件名： `08_jieba_custom_dict.py`

```
import jieba

# 准备一段包含多个专业词的文本
# 有些专业词 jieba 默认不一定能完整识别
text = "熟悉大数据开发、数据仓库、机器学习和自然语言处理"

# 先观察默认分词结果
# 这一步是为了让学生看到：不加自定义词时，专业词可能会被切碎
print("添加自定义词前：")
print(jieba.lcut(text))

# add_word 可以临时添加一个自定义词
# 添加后，jieba 会更倾向于把这些词作为整体切出来
jieba.add_word("大数据开发")
jieba.add_word("数据仓库")
jieba.add_word("机器学习")
jieba.add_word("自然语言处理")

# 再观察添加自定义词之后的分词结果
# 对比前后结果，就能看出自定义词典的作用
print("添加自定义词后：")
print(jieba.lcut(text))
```

15.3 使用词典文件

新建 user_dict.txt：

大数据开发
数据仓库
机器学习
自然语言处理
SparkSQL

代码：

```
import jieba

# 加载用户自定义词典
# user_dict.txt 中每一行可以写一个希望 jieba 识别的专业词
# 这种方式比反复写 jieba.add_word(...) 更适合项目维护
jieba.load_userdict("user_dict.txt")

# 准备待分词文本
text = "熟悉大数据开发、数据仓库、机器学习和自然语言处理"

# 加载自定义词典后再分词
print(jieba.lcut(text))
```

15.4 教学提示

自定义词典适合放入项目领域词汇，例如：

- 岗位名称
- 技能名称
- 证书名称
- 大数据组件名称

- AI 相关专业词

思考题：什么时候应该添加自定义词？

参考答案：

当你发现重要专业词被错误切分时，就应该考虑添加自定义词。例如 `自然语言处理` 被切成 `自然语言` 和 `处理`，但在项目中它应被视为一个完整技术方向，就可以加入自定义词典。

十六、综合案例：简历与岗位文本预处理

16.1 案例目标

把今天所学串起来：

1. 准备简历和岗位文本。
2. 使用 jieba 分词。
3. 过滤标点和停用词。
4. 标准化技能词。
5. 输出用于后续匹配计算的词列表。
6. 初步计算共同关键词。

16.2 代码清单

文件名： `09_resume_job_preprocess_case.py`

运行本案例前，请先准备 `stopwords.json`：

["的", "了", "和", "与", "或", "以及", "熟悉", "掌握", "具有", "相关", "经验", "负责", "参与", "能够", "进行", "岗位", "要求", "有", "做过"]

```
import json
import jieba
```

```
# 标点符号集合
```

```
# 这些符号不应该进入后续关键词匹配
```

```
PUNCTUATIONS = {
    ", ", "。", "\ ", "; ", ": ", "! ", "? ",
    ", ", ". ", "; ", ": ", "! ", "? ",
    "(", ")", " (", ") ", "[", "]", " 【", "】 ",
    " ", "\n", "\t"
}
```

```
# 技能词标准化映射表
```

```
# key: 文本中可能出现的写法
```

```
# value: 希望统一成的标准写法
```

```
# 例如 py、python 都统一成 Python
```

```
SKILL_MAPPING = {
    "py": "Python",
    "python": "Python",
    "Python语言": "Python",
    "spark": "Spark",
    "apache spark": "Spark",
    "sql": "SQL",
    "mysql": "MySQL",
    "hadoop": "Hadoop",
    "hive": "Hive",
    "机器学习": "Machine Learning",
    "深度学习": "Deep Learning",
    "自然语言处理": "NLP",
    "数据分析": "Data Analysis"
```

```
}
```

```
def load_stopwords(file_path):  
    """  
    从 stopwords.json 中读取停用词，并转换成 set。  
    """  
    # 读取 JSON 文件  
    # stopwords.json 中保存的是一个数组，例如：  
    # ["的", "了", "和", "熟悉", "掌握"]  
    with open(file_path, "r", encoding="utf-8") as f:  
        words = json.load(f)  
  
    # 转成 set 后，后面判断某个词是否为停用词会更快  
    return set(words)
```

```
def normalize_skill(word):  
    """  
    技能词标准化。  
    """  
    # 去掉词语两侧空白  
    word = word.strip()  
  
    # 英文技能词可能有大小写差异  
    # lower_word 用来做大小写不敏感匹配  
    lower_word = word.lower()  
  
    # 如果原词能直接匹配映射表，就返回标准写法  
    if word in SKILL_MAPPING:  
        return SKILL_MAPPING[word]
```

```
# 如果原词不能匹配，就尝试使用小写形式匹配
if lower_word in SKILL_MAPPING:
    return SKILL_MAPPING[lower_word]
```

```
# 如果映射表中没有这个词，就原样返回
return word
```

```
def preprocess_text(text, stopwords):
    """
```

文本预处理函数。

输入：原始文本

输出：清洗、标准化、去重后的词列表

```
"""
```

```
# 第一步：对原始文本进行分词
```

```
words = jieba.lcut(text)
```

```
# result 保存最终结果
```

```
result = []
```

```
for word in words:
```

```
# 第二步：去掉词语两侧空格
```

```
word = word.strip()
```

```
# 第三步：过滤空字符串
```

```
if not word:
```

```
    continue
```

```
# 第四步：过滤标点符号
```

```
if word in PUNCTUATIONS:
```

```
    continue
```

```
# 第五步：过滤停用词
# 例如“熟悉”“掌握”“岗位”“要求”等弱信息词
if word in stopwords:
    continue

# 第六步：技能词标准化
# 例如 py -> Python, 数据分析 -> Data Analysis
word = normalize_skill(word)

# 第七步：去重并保留顺序
# 同一个词在一段文本里出现多次时，只保留一次
if word not in result:
    result.append(word)
```

```
# 返回清洗后的词列表
return result
```

```
def calculate_overlap(list_a, list_b):
    """
    计算两个词列表的交集。
    这里先不计算复杂分数，只观察共同技能。
    """
    # 先把列表转换为集合
    # 集合之间可以直接用 & 求交集
    set_a = set(list_a)
    set_b = set(list_b)

    # set_a & set_b 表示两个集合中共同拥有的词
    return list(set_a & set_b)
```

```
if __name__ == "__main__":
    # 读取停用词配置
    # 运行前要保证当前目录下存在 stopwords.json
    stopwords = load_stopwords("stopwords.json")

    # 准备一份候选人简历
    # 这里先用字典模拟真实项目中的一行简历数据
    resume = {
        "resume_id": "R001",
        "name": "张三",
        "text": "熟悉py、Spark、SQL，做过数据分析项目，了解机器学习。"
    }

    # 准备多个岗位
    # 真实项目中，这些数据通常来自 jobs.csv 或数据库
    jobs = [
        {
            "job_id": "J001",
            "job_title": "数据分析师",
            "text": "岗位要求掌握Python编程，熟悉SQL，有数据分析经验。"
        },
        {
            "job_id": "J002",
            "job_title": "大数据工程师",
            "text": "岗位要求熟悉Hadoop、Hive、Apache Spark，有大数据处理经验。"
        },
        {
            "job_id": "J003",
            "job_title": "Java后端工程师",
            "text": "岗位要求掌握Java、Spring Boot、MySQL，有后端开发经验。"
        }
    ]
```



```
# 先处理简历文本
# 同一份简历会和多个岗位比较，所以先处理一次即可
resume_words = preprocess_text(resume["text"], stopwords)

# 打印简历处理结果
print("=" * 80)
print("候选人: ", resume["name"])
print("简历原文: ", resume["text"])
print("简历预处理结果: ", resume_words)

# 逐个处理岗位，并计算该岗位与简历的共同关键词
for job in jobs:
    # 对当前岗位文本做预处理
    job_words = preprocess_text(job["text"], stopwords)

    # 计算简历词列表和岗位词列表的交集
    overlap = calculate_overlap(resume_words, job_words)

    # 打印当前岗位的匹配观察结果
    print("=" * 80)
    print("岗位名称: ", job["job_title"])
    print("岗位原文: ", job["text"])
    print("岗位预处理结果: ", job_words)
    print("共同关键词: ", overlap)
```

16.3 结果解读

你应该能观察到：

- 数据分析师岗位和简历有 Python 、 SQL 、 Data Analysis 等共同词。

- 大数据工程师岗位和简历有 `Spark` 等共同词。
- Java 后端工程师岗位和这份简历的共同词较少。

这说明，我们已经能初步看出哪类岗位和简历更接近。

下一节课会继续学习：

- 词袋模型
- TF-IDF
- 余弦相似度

也就是把“共同关键词观察”升级为“可计算的相似度分数”。

十七、课堂任务

任务 1：补充停用词

请在 `stopwords.json` 中增加 5 个你认为信息量不高的词，并观察结果变化。

示例：

```
["项目", "基础", "方向", "工作", "使用"]
```

参考答案：

这些词是否应该加入，要看项目目标。如果它们在大多数简历和岗位中都频繁出现，而且不能明显区分岗位类型，可以加入停用词表。但如果某些词对岗位判断有帮助，比如 `项目` 可以体现实践经验，就不要轻易过滤。

任务 2：补充技能词映射

请在 `SKILL_MAPPING` 中增加至少 5 个映射。

示例：

```
# 把 PySpark 的不同写法统一成 PySpark
"pyspark": "PySpark",
"Pyspark": "PySpark",

# 把 scikit-learn 的常见简称统一成标准库名
"sklearn": "scikit-learn",
"Scikit Learn": "scikit-learn",

# 把中文技能词统一成英文标准写法
"数据库": "Database"
```

参考答案：

合理。 `pyspark` 和 `Pyspark` 应统一为 `PySpark`； `sklearn` 和 `Scikit Learn` 应统一为 `scikit-learn`； `数据库` 可以统一为 `Database`，方便和英文岗位要求对齐。

任务 3：增加一份简历和两个岗位

请自己添加：

- 一份 AI 方向简历
- 一个算法工程师岗位
- 一个数据库工程师岗位

参考答案示例：

```
# 新增一份 AI 方向简历
# text 字段是后续要进行分词和关键词提取的核心文本
resume = {
    "resume_id": "R002",
    "name": "李四",
    "text": "熟悉机器学习、深度学习、Python和PyTorch，做过图像分类项目。"
}

# 新增两个岗位，用来观察简历更偏向哪个岗位
jobs = [
    {
        # 算法工程师岗位和简历有较多共同技术词
        "job_id": "J004",
        "job_title": "算法工程师",
        "text": "要求熟悉Python、机器学习、深度学习，有模型训练经验。"
    },
    {
        # 数据库工程师岗位主要关注 MySQL、SQL 优化、数据库设计
        # 它和这份 AI 简历的共同词应该较少
        "job_id": "J005",
        "job_title": "数据库工程师",
        "text": "要求熟悉MySQL、SQL优化、数据库设计和数据备份。"
    }
]
```

预期结果：

- 李四应该更匹配算法工程师岗位。
- 数据库工程师岗位共同关键词较少。

任务 4：封装成模块

将以下函数复制到 `preprocess.py`：

- `normalize_skill`
- `preprocess_text`
- `calculate_overlap`

然后在另一个文件中导入使用：

```
# 从 preprocess.py 文件中导入已经封装好的函数
# 这样主程序就不用重复写预处理代码
from preprocess import preprocess_text, calculate_overlap
```

参考答案：

这种做法是正确的。它能让预处理逻辑从主程序中分离出来，后续无论是本地版项目、Streamlit 页面，还是 PySpark 迁移版，都可以复用同一套文本预处理逻辑。

十八、本节小结

本节完成了简历岗位匹配项目的文本预处理部分。

请记住这条主线：

```
原始文本 -> 分词 -> 清理标点 -> 过滤停用词 -> 技能词标准化 -> 干净词列表
```

核心结论：

1. NLP 是让计算机处理人类语言的技术。
2. 中文文本需要先进行分词。
3. 分词结果中会混入标点、停用词和不统一的技能写法。
4. 停用词过滤可以减少噪声。
5. 技能词标准化可以让 `py`、`python`、`Python语言` 被识别为同一个技能。
6. 文本预处理结果将作为后续 TF-IDF 和余弦相似度的输入。

本节之后，你应该能把：

熟悉`py`、`Spark`、`SQL`，做过数据分析项目

转换为：

`["Python", "Spark", "SQL", "Data Analysis", "项目"]`

信息整理清楚了，后面的匹配计算才有意义。